

Algoritmos Paralelos

Discente: Bruno Victor Paiva da Silva

1. **Estude e apresente como as primitivas de paralelismo `spawn`, `sync`, `and` `parallel for` podem ser relacionadas com o padrão e modelo de programação OpenMP.**

No modelo do Cormen, `spawn` é como bifurcar a execução. Quando uma thread encontra um `spawn`, ela cria uma nova thread “filha” para executar uma sub-rotina específica. Consequentemente a thread que chamou o `spawn` não espera. Ela continua executando o código imediatamente após o `spawn`, em paralelo com a thread filha.

No OpenMP, a diretiva mais análoga a esse comportamento é a `#pragma omp task`.

O `sync` é a primitiva de sincronização. Ela força a thread “mãe” parar e esperar até que todas as tarefas “filhas” que ela spawnou tenham terminado suas execuções.

No OpenMP, a diretiva análoga que faz uma thread esperar por suas tarefas filhas diretas é a `#pragma omp taskwait`.

No modelo do Cormen, o `parallel for` é a nossa principal ferramenta para paralelismo de dados. A ideia é que temos um loop `for` onde cada iteração é independente das outras.

O `parallel for` é responsável por dividir essas iterações entre as threads disponíveis para que elas rodem ao mesmo tempo.

Em OpenMP, a diretiva que faz exatamente isso é a `#pragma omp for`.

2. **Escolha um dos algoritmos que já implementou nas listas anteriores que poderiam se beneficiar de paralelismo e implemente-os utilizando OpenMP.**

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <algorithm>
#include <omp.h>
```

```

void merge(std::vector<int> &A, std::vector<int> &B, long long
begin, long long middle, long long end) {
    long long i = begin;
    long long j = middle + 1;
    long long k = begin;

    while (i <= middle && j <= end) {
        if (A[i] <= A[j]) {
            B[k++] = A[i++];
        } else {
            B[k++] = A[j++];
        }
    }

    while (i <= middle) {
        B[k++] = A[i++];
    }

    while (j <= end) {
        B[k++] = A[j++];
    }

    for (long long idx = begin; idx <= end; idx++) {
        A[idx] = B[idx];
    }
}

void parallel_merge_sort(std::vector<int> &A, std::vector<int>
&B, long long begin, long long end, int depth = 0) {
    if (begin < end) {
        if (end - begin < 8192) {
            std::sort(A.begin() + begin, A.begin() + end + 1);
            return;
        }

        long long middle = begin + (end - begin) / 2;

```

```

        if (depth < 5) {
#pragma omp task shared(A, B)
            parallel_merge_sort(A, B, begin, middle, depth + 1);

#pragma omp task shared(A, B)
            parallel_merge_sort(A, B, middle + 1, end, depth + 1);

#pragma omp taskwait
        } else {
            parallel_merge_sort(A, B, begin, middle, depth + 1);
            parallel_merge_sort(A, B, middle + 1, end, depth + 1);
        }

        merge(A, B, begin, middle, end);
    }
}

int main(int argc, char *argv[]) {
    if (argc < 3) return 1;

    long long n = std::atoll(argv[1]);
    int threads = std::atoi(argv[2]);

    std::vector<int> A(n);
    std::vector<int> B(n);

    for (auto &v : A) {
        v = rand();
    }

    omp_set_num_threads(threads);

    double start = omp_get_wtime();

#pragma omp parallel

```

```

{
#pragma omp single
    parallel_merge_sort(A, B, 0, n - 1);
}

double end = omp_get_wtime();

std::cout << n << ", " << threads << ", " << (end - start) <<
"\n";

return 0;
}

```

3. Apresente uma análise experimental do algoritmo implementado na questão 2 para realizar suas medições (sugestão: utilizar o NPAD).

Teste feito no NPAD, em 1 nó computacional, na partição intel-512 e 128GB de memória RAM

```

[bvpdsilva@service0 algoritmos-paralelos]$ sbatch run.sh
Submitted batch job 1863682
[bvpdsilva@service0 algoritmos-paralelos]$ watch queue -u $USER
[bvpdsilva@service0 algoritmos-paralelos]$ █

```

Tabela com os tempo:

Tamanho (N)	Threads (p)	Exec 1	Exec 2	Exec 3	Exec 4	Exec 5
6*1.000.000	1	0,069136	0,0681854	0,0680914	0,0678932	0,0683067
	2	0,0373264	0,0374878	0,0375522	0,0372246	0,0374138
	4	0,0332266	0,0310136	0,0336691	0,0333768	0,0334338
	8	0,0234385	0,0203709	0,0236037	0,0230271	0,0242926
	16	0,0177453	0,0184207	0,0183826	0,0179613	0,0182487
	32	0,0153096	0,0150007	0,0151613	0,0152255	0,0153731
6*5.000.000	1	0,391483	0,39064	0,390151	0,390584	0,389737
	2	0,206747	0,206549	0,206764	0,206062	0,205881
	4	0,169537	0,167427	0,16704	0,178053	0,166881
	8	0,108334	0,123252	0,12617	0,124479	0,123288
	16	0,0875775	0,0851456	0,0872749	0,0883608	0,0900124
	32	0,0685919	0,0690662	0,0694883	0,0694754	0,0705139
6*10.000.000	1	0,829767	0,825616	0,829626	0,82966	0,829582
	2	0,443438	0,441767	0,443589	0,44274	0,443757
	4	0,373518	0,357042	0,357602	0,356479	0,375595
	8	0,233679	0,268397	0,251745	0,264318	0,260663
	16	0,189952	0,180587	0,190268	0,18936	0,184958
	32	0,143886	0,14711	0,146659	0,14855	0,146347

Tabela da mediana dos tempos:

Threads (p)	$N = 1.000.000$	$N = 5.000.000$	$N = 10.000.000$
1	0,068185	0,390584	0,829626
2	0,037414	0,206549	0,443438
4	0,033377	0,167427	0,357602
8	0,023439	0,124479	0,260663
16	0,018249	0,087578	0,189360
32	0,015226	0,069475	0,146659

Tabela do Speedup:

Threads (p)	$N = 1.000.000$	$N = 5.000.000$	$N = 10.000.000$
1	1,00x	1,00x	1,00x
2	1,82x	1,89x	1,87x
4	2,04x	2,33x	2,32x
8	2,91x	3,14x	3,18x
16	3,74x	4,46x	4,38x
32	4,48x	5,62x	5,66x

Tabela da Eficiência

Threads (p)	$N = 1.000.000$	$N = 5.000.000$	$N = 10.000.000$
1	100,0%	100,0%	100,0%
2	91,1%	94,5%	93,5%
4	51,0%	58,3%	58,0%
8	36,4%	39,3%	39,8%
16	23,4%	27,9%	27,4%
32	14,0%	17,6%	17,7%